2026-09-01

`foil`: paired performance benchmarking for R packages

Visruth Srimath Kandali
California Polytechnic State University, San Luis Obispo

1 Executive Summary

R package maintainers need to detect runtime and memory regressions before changes are merged or released, but ordinary development machines and CI runners are noisy and R-level memory accounting does not capture the full footprint of workloads that execute in native code or child processes. `foil` is a Rust CLI that addresses the revision-comparison problem by running clean baseline and candidate revisions as randomized pairs on the same machine, retaining those pairs in inference, adjusting for repetition position and execution order, and reporting the revision effect with uncertainty.

The funded work will extend the existing runtime measurement with process-tree peak-memory measurement on Linux and Windows, binary distribution, and a strong R integration layer. Linux will use cgroup memory tracking; Windows will use Job Objects. This approach allows for measuring memory used by native libraries, compiled extensions, threads, and contained child processes.

The project will publish prebuilt `foil` binaries for Linux, macOS, and Windows, together with an R package for running benchmarks and analyzing results. `foil` will also support opinionated initializations such as `foil init r-package`, which will write a template benchmark configuration and GitHub Actions workflow. Dependency installation, compilation, and other project-specific setup remain under the maintainer's control. Runtime comparison already exists and runs on macOS, Linux, and Windows—while memory measurement targets Linux and Windows.

`foil` will let R package maintainers run revision-level runtime and memory comparisons on ordinary development machines and CI runners without maintaining dedicated benchmark hardware.

Work is scheduled from January through June 2027, with Linux memory measurement and the result schema first, followed by distribution and R integration, benchmark overhead and model diagnostics, Windows memory measurement, and release stabilization.

Requested funding is $10,000 USD for development labor across five milestones.

2 Signatories

2.1 Project team

Visruth Srimath Kandali is the sole developer of `foil`. He is a Master's student in Statistics at California Polytechnic State University, San Luis Obispo, and a member of the Stan development team. His open-source work includes sustained development of `loo` as a Google Summer of Code contributor in 2025 and 2026. Relevant contributions to `loo` include testing and package infrastructure modernization and `touchstone`-based performance benchmarks. He also develops R packages such as `bootstrapper`, an opinionated package for setting up package infrastructure and CI workflows.

More information is available at visruth.com.

2.2 Consulted

3 The Problem

Performance regressions can pass unit tests and reach users as slower code or higher memory use. R package maintainers therefore need to be able to determine whether a specific change meaningfully altered performance. On ordinary development machines and CI runners, however, runtime measurements are noisy enough that revision-level changes can be difficult to distinguish from normal variation. One way to reduce that noise is to measure baseline and candidate revisions on the same machine, which reduces machine-to-machine variation (Laaber et al. 2019).

Memory measurement presents another problem. Many R packages perform substantial work in C, C++, Fortran, Rust, BLAS, OpenMP, or child processes, so R-level allocation accounting does not capture all memory used by the benchmarked workload. Maintainers of packages with compiled or parallelized workloads can therefore miss memory regressions even if R-level allocation measurements appear unchanged. Reliable revision-level runtime and memory comparisons would let maintainers detect and investigate these regressions before they reach users, without requiring dedicated benchmarking hardware.

Existing R tools address parts of this problem. `bench` measures runtime and R allocation behavior (Hester and Vaughan 2025), `cross` runs expressions across package versions or Git branches (Vaughan 2026), and `atime` compares time and memory across versions and revisions (Hocking 2026; Amoakohene et al. 2026). `touchstone` comes closest to a paired revision experiment (Walthert and Wujciak-Jens 2026), and an August 19, 2026 search found `touchstone` configuration in 35 public repositories across 22 owners. However, these tools (as reviewed in Section 7) do not combine paired revision-level inference with memory measurement that includes work performed outside R's allocator.

Existing tools also vary in how they treat uncertainty. Some report point estimates or intervals without preserving the paired design in inference, while others use significance tests to classify a change. A core principle of `foil` is to preserve uncertainty rather than collapse it into a changed/unchanged decision. `foil` therefore centers on reducing and quantifying uncertainty in estimating the revision effect, leaving the practical importance of the change to the maintainer (Gelman and Stern 2006; McShane et al. 2019).

4 The proposal

4.1 Overview

`foil` is designed to evaluate whether a change made software slower or more memory-hungry—most naturally in the form of comparing the performance of a feature branch to `main`. The existing CLI checks out clean baseline and candidate revisions, runs them as randomized pairs in the same job, preserves those pairs in inference, adjusts for repetition position and execution order, and reports the revision effect with uncertainty. This makes useful comparison possible on ordinary development machines and CI runners without a historical benchmark database or dedicated benchmarking hardware.

The funded work adds process-tree peak-memory measurement, prebuilt binaries, and an R package. Process-tree memory matters for R because substantial workloads execute in C, C++, Fortran, Rust, BLAS, OpenMP, or child processes, outside R allocation accounting. `foil` will measure the benchmark process and its contained descendants while retaining native Linux and Windows metric semantics. Amongst the tools reviewed in Section 7, none combines arbitrary Git-revision commands, randomized paired measurement on ordinary CI runners, explicit repetition-position and execution-order adjustment, uncertainty on the revision effect, and OS process-tree peak memory.

From January through June 2027, the project will deliver Linux memory and the result schema, distribution and R integration, reduce `foil` overhead and present model diagnostics, Windows memory, and stable documented releases.

4.2 Detail

4.2.1 Model

`foil` models each paired repetition as a shared measurement level plus a candidate-minus-baseline effect. Both the shared level and the revision effect may vary with repetition position and run order, allowing the model to account for drift and order effects while estimating the adjusted difference between revisions. Pairing also accounts for additive disturbances shared by the baseline and candidate measurements. Section 9 gives the full model. `foil` reports the estimated revision effect together with its uncertainty rather than reducing the result to a binary significance decision (Gelman and Stern 2006; McShane et al. 2019).

4.2.2 Benchmark process-tree memory

Memory usage will be measured using OS facilities that track maximum memory usage across the benchmark process and its descendants. This captures memory used by native libraries, threads, and child processes. The exact semantics differ by OS: Linux will use cgroups, while Windows will use Job Objects. Memory measurements from Windows and Linux runs are not directly comparable because each platform measures a subtly different quantity. Linux will report per-invocation cgroup v2 `memory.peak`; Windows will report Job Object `PeakJobMemoryUsed` (Linux kernel developers 2026; Microsoft 2021). Section 8 goes into more detail.

4.2.3 R integration and distribution

The R package will be released on CRAN and the multiverse. It will install or locate a `foil` executable and provide a minimal R interface to `foil` to run benchmarks. It will also be able to import measurements and posterior draws for further analysis or visualizations. The Rust CLI will remain responsible for the core mechanisms, but the package will allow users to interact with it through R.

The Rust crate will be published on crates.io; prebuilt binaries and installers will be published through GitHub Releases using `dist`, with `cargo binstall` support. `foil init r-package` will write a starter `foil.toml`, R benchmark entry point, and GitHub Actions workflow.

4.2.4 Minimum Viable Product

The MVP is a complete Linux workflow for measuring a performance change—including memory—and making the result available in R. A user can install `foil` without a Rust toolchain, compare two Git revisions locally or in CI, measure runtime and process-tree peak memory, and obtain an adjusted revision effect with uncertainty. The R package can install or locate a compatible `foil` executable, run benchmarks, and import the resulting measurements and posterior draws. A generated project configuration and GitHub Actions workflow will reduce setup cost.

4.2.5 Architecture

The Rust CLI resolves revisions, creates clean worktrees, constructs the randomized paired schedule, runs benchmark commands, records observations, and performs inference. The benchmark command itself remains user-defined, so package-specific setup and the workload being measured stay under the maintainer's control. Runtime and memory measurements enter the same paired schedule and inference pipeline, while platform-specific backends handle the details of process containment and memory accounting. `foil` fits a model to estimate the revision effect and performs a Bayesian bootstrap to quantify uncertainty without specifying an error distribution (Rubin 1981).

The CLI writes measurements and inference results out to a machine-readable result format. The R package sits on top of this interface: it installs or locates a compatible executable, invokes `foil`, and reads the resulting measurements and posterior output. Benchmark orchestration, measurement, and statistical inference therefore have a single implementation in Rust, whether `foil` is used directly from the command line, in CI, or through R.

4.2.6 Assumptions

Linux process-tree memory requires usable cgroup v2 access, and Windows memory requires Job Object containment If access is unavailable, `foil` reports the metric as unsupported.

`foil` deliberately makes few modelling assumptions: drift and run-order effects are linear, paired repetitions are independent after accounting for those effects, and disturbances shared by baseline and candidate are additive.

4.2.7 External dependencies

`foil` depends on Git. Process-tree memory additionally requires cgroup v2 on Linux or Job Objects on Windows.

5 Project plan

5.1 Start-up phase

Development will build on the existing `foil` CLI, which already performs randomized paired runtime comparisons between Git revisions and reports the revision effect with uncertainty; the funded work adds process-tree memory measurement, prebuilt binaries, and an R package. Development will continue in the existing public GitHub repository. The existing code is flexible and was built with support for memory measurement in mind. Work will start immediately by evaluating cgroup

dependencies (e.g., `cgroups-rs`, `cgroups-fs`, `cgroup-memory`, or sticking with the hand-rolled interface) and implementation of memory measurement.

`foil` will remain dual-licensed under MIT OR Apache-2.0; the R package will use BSD-3. Development, issue tracking, releases, milestone summaries, and scope changes will remain public, with progress reported to the ISC at the midpoint and completion.

5.2 Technical delivery

- By February 28, 2027: finalize the process-tree memory boundary and result schema; implement the Linux cgroup v2 backend; add controlled process-tree fixtures and measurement tests.
- By March 31, 2027: publish prebuilt `foil` binaries and installers; implement the R package for binary installation or discovery, compatibility checks, benchmark invocation, and result import; add `foil init r-package`, a GitHub Actions example, and compact terminal interval plots.
- By April 30, 2027: improve diagnostics for repetition-position and run-order effects; measure the overhead introduced by `foil`, reduce avoidable overhead, and add benchmarks to track it.
- By May 31, 2027: implement the Windows Job Object backend and add containment tests.
- By June 30, 2027: resolve release blockers, complete methodology documentation, stabilize the result schema and R interface, and publish stable releases of `foil` and the R package.

Process-tree memory on macOS, parallel execution, and additional inference engines are outside the funded scope.

5.3 Other aspects

Code, documentation, issue tracking, and releases will remain public on GitHub.

An announcement post will invite maintainers to test early releases. Milestone posts will keep track of progress, and longer form midpoint and final posts will report the R workflow, process-tree memory work, benchmark overhead, and known limitations. Updates will be published on visruth.com, offered to the R Consortium blog, and submitted to R Weekly. Material scope changes will also be reported at ISC meetings.

5.4 Budget & funding plan

Requested funding is $10,000 USD for development labor. Under partial funding, Windows memory support will be cut first.

Milestone	Target	Expected outcome	Budget allocation
Linux process-tree memory	February 28, 2027	Memory contract and result schema; Linux cgroup v2 backend; controlled process-tree fixtures	$2,500

Milestone	Target	Expected outcome	Budget allocation
Distribution and R integration	March 31, 2027	Prebuilt releases and installers; R package with binary installation/discovery, compatibility checks, invocation, and result import; initializer and CI example	$2,500
Benchmark overhead and model diagnostics	April 30, 2027	Measured and reduced `foil` overhead; regression benchmarks and improved model diagnostics	$1,500
Windows process-tree memory	May 31, 2027	Windows Job Object backend with containment tests	$2,500
Stabilization and release	June 30, 2027	Stable CLI and R-package releases, methodology documentation, stabilized interfaces, and release blockers resolved	$1,000

6 Success

6.1 Definition of done

The project is complete when `foil` ships Linux and Windows process-tree memory backends, prebuilt binaries requiring no Rust toolchain, and an R package that can install or invoke a `foil` executable and read its results. The release will also include a machine-readable result format, `foil init r-package`, documented R package GitHub Actions workflows, and extended documentation on the software and methodology.

In essence, "done" is when `foil` measures both runtime and memory on Linux and Windows and is easy to set up with GitHub Actions for a typical R package.

6.2 Measuring success

Automated tests will verify the Linux and Windows memory backends and run them on controlled tasks to ensure measurement works. Benchmarks will track overhead introduced by `foil`, while model diagnostics will expose repetition-position and run-order effects. Release checks will verify that prebuilt binaries can be installed without a Rust toolchain, that the R package can locate or

install a compatible `foil` executable, run benchmarks, and read the machine-readable results, and that `foil init r-package` produces a working GitHub Actions workflow.

All results will remain public on GitHub.

6.3 Future work

Potential future work includes process-tree memory on macOS, parallel benchmark execution, additional statistical models, historical comparisons, and a tool similar to `git bayesect` for locating performance regressions across commits (Jain 2026).

References

Amoakohene, Doris, Anirban Chetia, Igor Steinmacher, and Toby Hocking. 2026. "The r Journal: Asymptotic Benchmarking Using the Atime Package." *The R Journal* 18: 169–87. https://doi.org/10.32614/RJ-2026-013.

curl project. 2026. "curl Performance Tests." https://github.com/curl/perf.

Gelman, Andrew, and Hal Stern. 2006. "The Difference Between 'Significant' and 'Not Significant' Is Not Itself Statistically Significant." *The American Statistician* 60 (4): 328–31. https://doi.org/10.1198/000313006X152649.

Hester, Jim, and Davis Vaughan. 2025. *bench: High Precision Timing of R Expressions*. https://bench.r-lib.org/.

Hocking, Toby. 2026. *atime: Asymptotic Timing*. https://doi.org/10.32614/CRAN.package.atime.

Jain, Shantanu. 2026. "Git Bayesect." March 22. https://hauntsaninja.github.io/git_bayesect.html.

Laaber, Christoph, Joel Scheuner, and Philipp Leitner. 2019. "Software Microbenchmarking in the Cloud. How Bad Is It Really?" *Empirical Software Engineering* 24 (4): 2469–508. https://doi.org/10.1007/s10664-019-09681-1.

Linux kernel developers. 2026. "Control Group V2." https://docs.kernel.org/admin-guide/cgroup-v2.html.

McShane, Blakeley B., David Gal, Andrew Gelman, Christian P. Robert, and Jennifer L. Tackett. 2019. "Abandon Statistical Significance." *The American Statistician* 73 (sup1): 235–45. https://doi.org/10.1080/00031305.2018.1527253.

Microsoft. 2021. "JOBOBJECT_EXTENDED_LIMIT_INFORMATION Structure." April 2.https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-jobobject_extended_limit_information .

Microsoft. 2025. "Job Objects."https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects .

Rdatatable. 2026. “data.table.” https://github.com/Rdatatable/data.table.

Rubin, Donald B. 1981. “The Bayesian Bootstrap.” *The Annals of Statistics* 9 (1): 130–34. https://doi.org/10.1214/aos/1176345338.

Stenberg, Daniel. 2026. “curl Performance.” August 14. https://daniel.haxx.se/blog/2026/08/14/curl-performance-2/.

Tandon, Akash, and Toby Dylan Hocking. n.d. “Rperform: Performance Testing for R Packages.” https://github.com/analyticalmonk/Rperform.

Vaughan, Davis. 2026. *cross: Run Expressions Across Package Versions*. https://github.com/DavisVaughan/cross.

Walthert, Lorenz, and Jacob Wujciak-Jens. 2026. *touchstone: Continuous Benchmarking with Statistical Confidence Based on 'Git' Branches*. https://github.com/lorenzwalthert/touchstone.

7 Appendix A: Existing tools and evidence of demand

7.1 Evidence of demand

The R community has previously invested directly in revision-level performance testing. `Rperform`, developed through Google Summer of Code projects in 2015, 2016, and 2022, was designed to track runtime and memory across Git revisions and branches and integrate performance testing into package CI. `Rperform` is no longer actively developed, but this repeated investment shows interest in detecting performance changes during R package development (Tandon and Hocking, n.d.).

An August 19, 2026 GitHub search for `touchstone` in `.Rbuildignore` files returned 35 public repositories across 22 owners after excluding repositories from the `touchstone` authors and the grant author. The repositories span statistical computing, epidemiology, Stan, graph analysis, simulation, and other R projects. Further, `data.table` uses `atime` to investigate timing and memory differences (Rdatatable 2026; Hocking 2026; Amoakohene et al. 2026). Looking beyond R, the `curl` project maintains dedicated performance-regression infrastructure because shared cloud runners proved too variable for stable comparisons (Stenberg 2026; curl project 2026).

ℹ `Nushell` GitHub search

```
gh search code touchstone --filename .Rbuildignore --limit 1000 --json repository |
  from json |
  get repository.nameWithOwner |
  uniq |
  where $it !~ 'lorenzwalthert|assignUser|VisruthSK' |
  sort
```

7.2 Comparison with adjacent tools

`foil` is directly inspired by `touchstone`, which randomizes the two branches within each block and records the block for every observation (Walthert and Wujciak-Jens 2026). Its current confidence-interval implementation fits `elapsed ~ branch`, so the recorded pairing and execution order do not enter the fit.

Memory is a second distinction. R allocation counters do not capture all memory used outside R's allocator, including compiled code and child processes. `foil` will measure OS-level peak memory for the benchmark process and its contained descendants. Section 8 defines the platform-specific quantities.

Tool	Pair modelled	Order/drift handling	Effect uncertainty	Memory	Workload
`foil`	**Yes**	**Repetition position and execution order modeled**	**Posterior distribution**	**OS process-tree peak**	**Arbitrary command**
`bench`	No	None	No	R allocations and GC	R expression
`atime`	No	None	No	Asymptotic memory usage (`bench`)	R expression (`bench`)
`cross`	No	None	No	R allocations and GC (`bench`)	R expression (`bench`)
`touchstone`	No	Randomized order, not modeled	**Confidence interval**	No	R expression
`Rperform`	No	None	No	Sampled R-process RSS	R package test files
`tango`	**Yes**	Randomized order, not modeled	Significance test	No	Rust microbenchmark
`zenbench`	**Yes**	Drift diagnosed, not modeled; execution order not modeled	**Confidence interval**	Allocations	Rust microbenchmark
CodSpeed	No	Controlled execution environment	Impact and regression thresholds	Allocations	Supported integrations/CLI

Tool	Pair modelled	Order/drift handling	Effect uncertainty	Memory	Workload
Bencher	No	None	Statistical thresholds and alerts	Varies	Varies

`touchstone` is the closest R workflow, while `tango` and `zenbench` are the closest paired statistical designs. None of the tools reviewed combines arbitrary command revision comparison, pairing retained in inference, explicit adjustment for repetition position and execution order, effect uncertainty, and OS process-tree peak memory.

8 Appendix B: Process-tree memory semantics

8.1 Measurement boundary

A measured invocation begins with the benchmark command and includes descendants created during that invocation. `foil` setup and teardown and any pre-existing processes remain outside the boundary. Native libraries and threads contribute through the measured process; parallel workers created as child processes contribute when they remain inside the platform containment boundary. Linux and Windows use different OS mechanisms to implement the same process-tree measurement boundary.

The posterior and summary code is already generic over the measured metric and can carry peak-memory values; benchmark runs currently record elapsed time only. The funded work adds the OS backends specified below.

8.2 Implementation details

For each measured invocation on Linux, `foil` will start the benchmark in a new cgroup, and read `memory.peak` after the command and measured descendants complete. Linux defines `memory.peak` as the maximum memory usage recorded for a cgroup and its descendants (Linux kernel developers 2026). If the required cgroup v2 access is unavailable, the metric is unsupported.

On Windows, `foil` will associate the benchmark with a Job Object before the measured workload proceeds and will report `PeakJobMemoryUsed` (Microsoft 2021) after the command and contained descendants complete. Child processes normally inherit job membership; nested jobs and breakaway settings can change containment (Microsoft 2025). Again, if unsupported, `foil` will only record runtime.

9 Appendix C: Statistical model

For repetition i, let B_i and C_i be the baseline and candidate measurements, r_i the centered repetition position, and $o_i \in \{-1, +1\}$ the run-order contrast indicating which branch was run first. Let $z_B = -1$ and $z_C = +1$ identify the baseline and candidate measurements within each pair.

A simple model for the two measurements is

$$y_{ik} = \mu + \frac{z_k}{2}\left(\Delta + \Delta_r r_i + \Delta_o o_i\right) + \varepsilon_{ik}.$$

Here, μ is the midpoint of the baseline and candidate measurements. The Δ coefficients describe the candidate-minus-baseline effect. Δ is the effect at $r = o = 0$, while Δ_r and Δ_o describe how that effect changes with repetition position and run order.

The 1/2 makes the Δ coefficients direct candidate-minus-baseline contrasts. In particular,

$$E[C_i - B_i] = \Delta + \Delta_r r_i + \Delta_o o_i.$$

The full model also allows the level shared by the baseline and candidate to vary with repetition position and run order:

$$y_{ik} = \mu + \mu_r r_i + \mu_o o_i + \frac{z_k}{2}\left(\Delta + \Delta_r r_i + \Delta_o o_i\right) + \varepsilon_{ik}.$$

The μ_r and μ_o coefficients are nuisance terms. They account for changes that affect the baseline and candidate equally, while the corresponding Δ coefficients describe changes in the candidate-minus-baseline effect.

`foil` fits this model through its equivalent midpoint and difference regressions. Define

$$m_i = \frac{B_i + C_i}{2}, \qquad d_i = C_i - B_i.$$

Then

$$m_i = \mu + \mu_r r_i + \mu_o o_i + \varepsilon_i^{(m)},$$

and

$$d_i = \Delta + \Delta_r r_i + \Delta_o o_i + \varepsilon_i^{(d)}.$$

Thus the midpoint regression estimates the shared level, while the difference regression estimates the candidate-minus-baseline effect. A disturbance shared by both measurements in a pair cancels from d_i.

For each Bayesian-bootstrap draw, `foil` samples one weight for each pair,

$$w_i \sim \mathrm{Exp}(1),$$

and fits both regressions by weighted least squares using the same weights. Normalizing these weights gives

$$\left(\frac{w_1}{\sum_j w_j}, \ldots, \frac{w_n}{\sum_j w_j}\right) \sim \mathrm{Dirichlet}(1, \ldots, 1),$$

so this is the standard Bayesian bootstrap (Rubin 1981). Using the same pair weights for both regressions preserves their joint uncertainty.

The fitted intercepts give the adjusted baseline and candidate values at $r = o = 0$:

$$\mu_B = \hat{\mu} - \frac{\hat{\Delta}}{2}, \qquad \mu_C = \hat{\mu} + \frac{\hat{\Delta}}{2}.$$

Shrinkage, when requested, acts only on the candidate-minus-baseline effect. The parameter λ represents a prior count of zero difference pseudo-observations. For $\lambda = 0$, the model above is

unchanged. Since each observed pair has expected weight $E[w_i] = 1$ while the pseudo-observation has expected weight $E[g] = \lambda$, λ can be interpreted as the effective prior sample size for zero difference. For $\lambda > 0$, each Bayesian-bootstrap draw additionally samples

$$g \sim \text{Gamma}(\lambda, 1)$$

and adds a zero difference pseudo-observation at $r = o = 0$ with weight g. Equivalently, the difference regression minimizes

$$\sum_i w_i \left(d_i - \Delta - \Delta_r r_i - \Delta_o o_i\right)^2 + g\Delta^2$$

while the midpoint regression is unaffected.